Université de Haute-Alsace
École Nationale Supérieure d'Ingénieurs Sud Alsace

Informatique et Réseaux - 1A

Algorithm and Data Structures

TD2 : Search and Sort Algorithms

Exercise 1

— The Lost Artifact

An archaeologist has discovered n ancient artifacts, each identified by a unique integer ID. The IDs are stored in an array A of size n, which may or may not be sorted. Your task is to find the index of a target ID x, or return -1 if it is absent.

Input. An integer n, an array A of n distinct integer IDs, and a target integer x.

Output. The index i such that $A[i] = x$, or -1 if no such index exists.

Constraints.

$$1 \leq n \leq 10^6, \qquad -10^9 \leq A[i], x \leq 10^9.$$

Indices start at 0.

Questions.

1. **Preparing the search system.** Implement `is_sorted_increasing`, which returns whether the array is strictly increasing.
2. **Linear search.** Implement a search that examines the elements sequentially. Determine its worst-case time complexity.
3. **Jump search.** For a sorted array, jump forward in blocks of size $\lfloor \sqrt{n} \rfloor$, then search sequentially inside the relevant block. Implement this algorithm and determine its worst-case time complexity. First verify that the array is strictly increasing; otherwise return -1.
4. **Binary search.** Implement **iterative** binary search by repeatedly eliminating half of the remaining interval. Determine its worst-case time complexity. First verify that the array is strictly increasing; otherwise return -1.
5. **Performance comparison.** Generate sorted arrays of increasing sizes and compare the execution times of the three algorithms in **seconds**, using `clock()`[1]. Discuss the effect of sortedness verification on the measurements.

Exercise 2

— The Tournament Rankings

A programming tournament has just ended! Each of the n participants has obtained an integer score, but the results have been recorded in the order in which participants finished their challenges.

Before announcing the winners, the organizers must sort the scores in **nondecreasing order**.

[1] C documentation: `https://devdocs.io/c/chrono/clock`

Your task is to implement and compare different sorting algorithms to help the organizers rank the participants.

Input.

- An integer n, the number of participants.
- An array A containing n integer scores.

Output.

Rearrange the elements of A so that:

$$A[0] \leq A[1] \leq \cdots \leq A[n-1].$$

Constraints.

$$1 \leq n \leq 10^5, \qquad 0 \leq A[i] \leq 10^6.$$

Multiple participants may obtain the same score.

Questions.

1. **Preparing the sorting system.**

   Before implementing the sorting algorithms, implement the following utility functions:

   - `swap_int`: exchanges the values of two integers.
   - `is_sorted_ascending`: checks whether an integer array is sorted in ascending order.
   - `copy_array`: copies the contents of one integer array into another.

2. **Selection sort.**

   The organizers first decide to repeatedly select the participant with the lowest remaining score and place that score at its correct position.

   Implement **selection sort** using an **iterative** approach.

   Determine its worst-case time complexity.

3. **Insertion sort.**

   Another organizer proposes processing the scores one by one, inserting each new score into its correct position among the previously sorted scores.

   Implement **insertion sort** using an **iterative** approach.

   Determine its best-case and worst-case time complexities.

4. **Bubble sort.**

   A third organizer suggests repeatedly comparing neighboring scores and swapping them whenever they are in the wrong order.

   Implement **bubble sort** using an **iterative** approach.

   Determine its worst-case time complexity.

5. **Merge sort.**

   As the tournament grows, the organizers need a more efficient sorting algorithm.

   One strategy is to divide the scores into two halves, sort each half independently, and then merge the two sorted halves.

   Implement **merge sort** using a **recursive** approach.

   Determine its worst-case time complexity.

6. **Quick sort.**

   Another strategy consists of selecting a pivot, partitioning the scores around it, and recursively sorting the resulting partitions.

   Implement **quick sort** using a **recursive** approach.

   Determine its best-case and worst-case time complexities.

7. **Performance comparison.**

   Generate arrays of increasing sizes containing randomly generated scores.

   For each array, compare the execution times of the five sorting algorithms.

   Measure execution times in **seconds** using `clock()`[2].

   To ensure a fair comparison, each algorithm must receive an identical copy of the original unsorted array.

   Compare your experimental observations with the theoretical time complexities.

[2] C documentation: `https://devdocs.io/c/chrono/clock`